ASCEND UAV

Autonomous Drone System for GPS-Denied Area Coverage

Final Round Proposal Report

by
Team Name: AVIATORS

Team ID: 10270
July 2026
Institute Name: RGUKT, Nuzvid, 521202

Contents

Abhyudaya- Autonomous Drone System for GPS-Denied Area Coverage

1 Introduction

ISRO's initiative to advance autonomous surveyor technologies through swarm expeditions for planetary exploration is a significant step forward. Autonomous systems capable of collaborative exploration, navigation, and dynamics have gained considerable attention in space applications in recent years. We appreciate ISRO's efforts in encouraging students to develop innovative frameworks that address the requirements for future interplanetary missions. Since the IRoC-U 2026 challenge, titled "Towards Swarm Expedition: Autonomous Surveyor Challenge for Exploration, Navigation and Dynamics (ASCEND)", emphasizes autonomous navigation, landing, power transfer, and image data validation without external aids, this report presents our proposed framework designed to meet the specifications outlined in the IRoC-U 2026 preliminary rulebook. The use of satellite-based navigation systems, external markers, or local positioning systems remains strictly prohibited. Therefore, we explore alternative methods such as visual odometry, inertial sensors, and potentially coordination algorithms for navigation and control.

The Autonomous Surveyor Challenge for Exploration, Navigation and Dynamics(ASCEND) system consists of two integrated units: a fully equipped quadcopter and a ground-based station designed to perform navigation, communication, docking, and battery charging operations. The navigation subsystem enables the Unmanned Aerial Vehicle(UAV) to perform autonomous flight in both outdoor and GPS-denied environments. Multiple onboard sensors, including inertial measurement units (IMUs), cameras, and ranging sensors to continuously estimate the UAV's position, orientation, and velocity. Sensor-fusion algorithms combine these measurements to generate a reliable estimate of the UAV's state, allowing stable flight, waypoint tracking and precision landing in GPS-denied environments.

Communication between the UAV and the ground station is established through a wireless telemetry link that supports bidirectional data transfer. In addition to command and control data, the communication system is designed for high-throughput transfer of images and sensor data collected during missions for enabling edge computing.

The onboard battery-management system continuously monitors battery voltage, current, and remaining capacity throughout the mission. When the battery level falls below a predefined threshold, or the assigned task is completed, the UAV autonomously initi-

ates a return-to-base procedure. The docking process is achieved through a precision-landing mechanism that guides the UAV toward the charging station using navigation feedback and visual alignment techniques. As the UAV approaches the docking platform, fine-position corrections are performed to align the charging contacts accurately. Once docking is confirmed, electrical contact is established between the charging pads on the UAV and the base station.

After successful docking, the charging subsystem begins controlled battery charging while continuously monitoring key electrical parameters to ensure safe operation. The ground station supervises the charging cycle, performs battery-health checks, and prepares the UAV for subsequent missions. Simultaneously, mission data collected during flight can be transferred and processed at the base station.

By integrating autonomous navigation, wireless communication, precision docking, and intelligent charging within a unified architecture, the ASCEND platform demonstrates a self-sustaining UAV ecosystem capable of executing repeated missions with minimal human intervention.

2 Navigation and Control

2.1 Navigation Overview

The proposed autonomous UAV system performs boundary-based navigation and area coverage using computer vision, waypoint generation, and onboard flight control. The navigation framework is designed to detect and follow a yellow boundary line, generate a closed-loop map of the arena, and perform systematic coverage of the enclosed area before returning and landing precisely on the base station.

Objectives The navigation subsystem is designed to satisfy three primary mission requirements:

- **Arena survey** – autonomously locate and trace the yellow boundary line to reconstruct the geometry of the arena.
- **Complete, boundary-safe coverage** – sweep the entire enclosed area with minimal overlap while remaining strictly within the detected boundary.
- **Precise return and landing** – return to the takeoff location and achieve accurate landing on the base station.

These objectives are fulfilled through the combined use of two independent sources, namely the vision system and the inertial system. Real-time visual perception, obtained from the onboard RGB camera, is used continuously throughout the mission, such as for

boundary detection during the spiral scan, centroid- and grid-based tracking during line following, waypoint generation during coverage planning, and marker detection during the final landing approach. In parallel, the Pixhawk flight controller supplies attitude and position estimation (via its IMU, magnetometer, barometer, and optical flow sensor), which the onboard controller uses to navigate to the commanded setpoints and maintain stable flight.

Usage of Visual Initial Odometry(VIO) instead of SLAM A deliberate architectural decision in this system is that the RGB camera stream and the Pixhawk's internal state estimate are never fused into a joint estimator no Simultaneous Localisation and Mapping (SLAM) module is employed. Vision is used exclusively for decision-making (line detection, centroid computation, grid-based reasoning), while all position holding and trajectory execution rely solely on the Pixhawk's internal state estimate, which itself incorporates a visual-inertial contribution through the optical flow sensor. This is different from conventional vision-guided UAV navigation architectures, which typically construct an occupancy map through continuous visual-inertial fusion. In this system, the boundary map is instead represented as the ordered sequence of North-East-Down(NED) waypoints recorded during line following, a substantially lighter-weight alternative to a SLAM-derived map.

This design choice reduces onboard computational load, avoids the calibration and tuning overhead associated with a full sensor-fusion pipeline, and remains sufficient for the mission requirements: a boundary geometry for coverage planning, and a stable flight controller for execution. The associated trade-off gradual position drift accumulated over the mission duration is addressed algorithmically rather than through continuous fusion. At loop closure, the drone's internally tracked position is compared against the Pixhawk telemetry position, and the resulting offset is applied as a correction to both the coverage waypoints and the return-home target. Section 5 quantifies this drift behaviour experimentally and extends the same correction principle to surveyed ground-truth features.The overall navigation logic executed by the UAV is summarised in Algorithm.

Algorithm 1 Autonomous Boundary Navigation and Coverage

Require: Takeoff position (N_0, E_0, D_0), target flight height H, HSV threshold $[T_{low}, T_{high}]$
Ensure: Complete arena coverage and precision landing at base station

Establish MAVSDK connection, arm UAV, engage offboard mode
Ascend to hover altitude $D_0 - H$; stabilise sensors
Capture and lock mission yaw ψ_{lock} from telemetry
Phase 1 – Boundary Search:
while yellow line not detected **and** search radius $\leq R_{max}$ **do**
    Command outward spiral setpoint
    Acquire frame; apply HSV segmentation with $[T_{low}, T_{high}]$
end while
if line not detected **then**
    Return to (N_0, E_0) and land ▷ Abort
end if
Phase 2 – Alignment: Centre UAV on line centroid using lateral/yaw correction
Phase 3 – Boundary Following:
Initialise waypoint list $\mathcal{W} \leftarrow \{(N_0', E_0')\}$
repeat
    Classify active grid blocks (Forward, Back, Left, Right, Centre)
    Select motion primitive (forward, lateral, corner) based on active blocks
    Apply centroid-based lateral correction
    Execute setpoint; append current position to $\mathcal{W}$
until loop closure detected (distance to $\mathcal{W}_0 \leq d_{th}$) **or** line lost
Phase 4 – Drift Estimation: $\Delta N, \Delta E \leftarrow$ (telemetry position) − (tracked final waypoint)
if loop closed **and** $|\mathcal{W}| \geq W_{min}$ **then**
    Phase 5 – Coverage Planning:
    Derive boundary bounding box from $\mathcal{W}$ along $(\psi_{lock}, \psi_{lock} + 90°)$ axes
    Compute row count N and per-row waypoint count dynamically from boundary extent
    Apply drift correction $(\Delta N, \Delta E)$ to all coverage waypoints
    Execute Boustrophedon sweep row-by-row
end if
Phase 6 – Return Home: Apply drift correction; fly to corrected home position
Phase 7 – Precision Landing:
Climb to AprilTag scan altitude; activate tag-detection thread
while tag not centred **do**
    Estimate lateral offset from tag pose; apply yaw-corrected setpoint
end while
while altitude above home > blind-land threshold **do**
    Descend by fixed step; re-correct lateral position using tag offset
end while
Trigger flight-controller `land()`; confirm touchdown; disarm

- **Takeoff to 3 m and Hover:** The UAV connects to the Pixhawk via MAVSDK, arms automatically, switches to offboard mode, and ascends to a fixed altitude of

3 m using NED coordinate control. It then holds a stationary hover to allow sensor stabilization, camera exposure adjustment, and telemetry initialization before navigation begins.

- **Random Direction Exploration:** After stabilization, the UAV executes an outward spiral scanning trajectory from the takeoff position to search for the yellow boundary line, with the search radius progressively increasing for systematic coverage. The onboard camera captures and processes frames in real time throughout, and the search terminates once the boundary is detected or the maximum radius R_{max} is reached. This design is independent of initial heading of ASCEND. The UAV can start direction the boundaries when it placed at inital heading

- **Yellow Line Detection and Centering**

  To achieve accurate alignment with the boundary, the UAV first detects the yellow line using a Hue Saturation Value(HSV) coloUr segmentation approach. The yellow region is extracted from the camera image using the HSV threshold range of **(19, 140, 150)** to **(35, 255, 255)**. Morphological operations are then applied to reduce noise and improve the continuity of the detected line.

  After detection, the centroid of the yellow line is computed and used as a reference for alignment. The UAV continuously compares the centroid position with the center of the camera frame to determine the required motion corrections. Based on the calculated offset, the UAV performs lateral movements and yaw adjustments until the yellow line is centered within the image frame.

  This yellow line centering process ensures that the UAV is properly aligned with the boundary before initiating the boundary-following phase, thereby improving tracking accuracy and navigation reliability.

- **Yellow Line Following and Waypoint Recording**

  After alignment with the center of waypoint recording, the UAV begins autonomous boundary following while maintaining a fixed yaw orientation called as yaw-locked navigation, which preserves a consistent coordinate frame for the remainder of the mission. Rather than relying on the centroid alone, the follower interprets the yellow mask as a 3×3 grid of blocks (Forward, Back, Left, Right, and Centre) and reasons over the pattern of active blocks to select the appropriate motion primitive, including dedicated corner-handling logic. This hybrid grid-and-centroid formulation distinguishes the approach from a conventional single-centroid line-tracking controller. As shown experimentally in Section 5, the tolerance of this grid-based centering step is one of the two dominant sources of the lateral (cross-axis) positional drift observed across repeated trials.During boundary traversal, the current

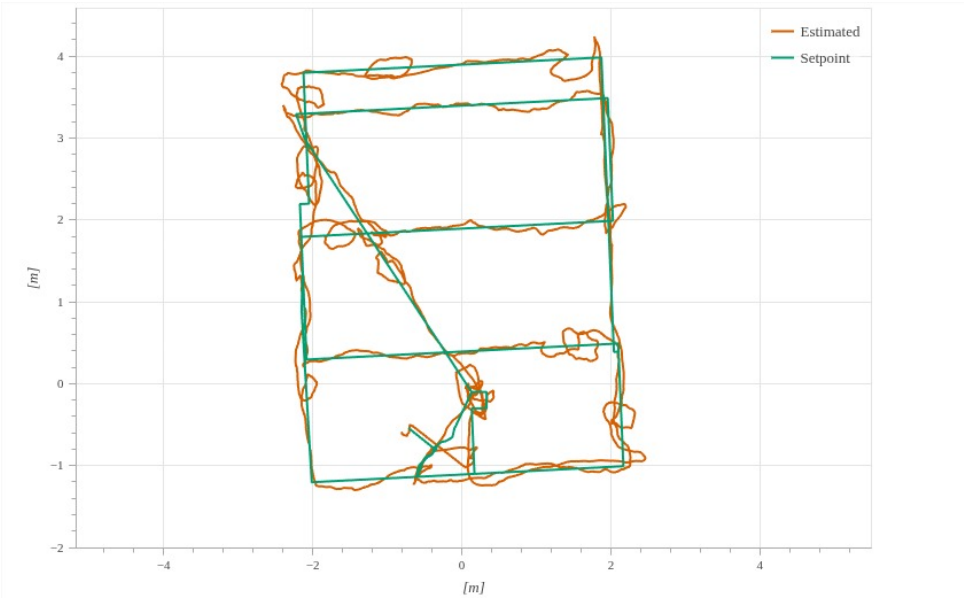

Figure 1: System architecture of the autonomous UAV platform.

NED position is continuously recorded as a waypoint following each executed move. This recorded waypoint sequence serves a dual purpose: it enables loop-closure detection, and, once the loop is closed, it directly constitutes the boundary polygon supplied to the Boustrophedon coverage planner.

- **Boundary Extraction**

  Frm figure 1,Boundary extraction is performed using the set of recorded waypoints generated during line following. Loop closure is detected by monitoring the Euclidean distance between the current waypoint and the initial waypoint. Once the distance falls below a threshold, the system declares successful loop closure.

  The resulting waypoint set forms a polygonal representation of the detected arena.

2.1.1 Boustrophedon Coverage Path Planning

From figure 1,Boustrophedon Coverage Path Planning (BCPP) [1] is employed to sweep the arena in an alternating back-and-forth pattern once the boundary polygon has been established. The recorded boundary is projected onto the locked heading axis and its perpendicular axis to obtain an effective length L and width W. Given an effective sweep width S, the number of coverage rows, total path length, and total covered area are given by:

$$N = \left\lceil \frac{W}{S} \right\rceil, \qquad D = N \times L, \qquad A = L \times W$$

Both the number of rows and the number of waypoints generated within each row are computed dynamically from the measured extent of the boundary polygon at runtime,

rather than fixed in advance. This allows the same implementation to adapt automatically to arenas of differing size or shape without manual re-tuning. Within each row, waypoints are distributed at equal intervals, ensuring uniform sampling along the sweep direction and preventing the uneven overlap or coverage gaps that fixed-interval waypoints would otherwise introduce on an arena of unknown dimensions. Because the row spacing here and the surveyed-feature setpoints discussed in Section 5 are both generated from the same measured boundary, any deviation between the recovered and true arena size propagates consistently to both the coverage path and the target markers.

The operational area is divided into parallel sweep lines, which the UAV traverses in an alternating zig-zag pattern, reversing direction and applying a lateral shift after each pass until the entire region has been covered.

Sweep Width Calculation The onboard Intel RealSense D455 is mounted at flight height $H = 2$ m with a horizontal field of view of $\theta = 86°$. The resulting ground footprint per frame is:

$$G = 2H \tan\left(\tfrac{\theta}{2}\right) = 2(2)\tan(43°) \approx 3.73 \text{ m}$$

A sweep step of $S = 1.0$ m was selected, yielding an inter-lane overlap of $O = G - S = 2.73$ m, equivalent to approximately **73.2% overlap**. This high overlap margin was selected deliberately to guarantee complete coverage with no unobserved gaps, provide multiple viewing angles of each region, and reduce the likelihood of missing small features or segments of the boundary line, at an acceptable cost of flight-path redundancy.

Implementation in ASCEND UAV The algorithm is implemented entirely through waypoint-based navigation, in which the UAV maintains stable altitude, heading, and velocity while stepping through the dynamically generated row waypoints, with continuous image capture synchronised to motion for terrain observation.

Advantages Because both the row count and per-row waypoint density are derived directly from the measured boundary rather than fixed a priori, the implementation generalises across arenas of different dimensions without modification, while retaining the deterministic, low-complexity, and predictable behaviour that makes Boustrophedon coverage well suited to real-time UAV operation.

2.1.2 AprilTag-Based Precision Landing

Precision landing on the base station is achieved through AprilTag detection rather than GPS-only positioning. Following the coverage sweep and return-home manoeuvre, the UAV climbs to a fixed hover altitude above the recorded home position and initiates a

dedicated detection thread that searches for a designated AprilTag (family `tag36h11`, fixed target ID) mounted on the base station.

Upon detection, the tag pose is used to compute a lateral offset between the tag centre and the image centre; this offset is transformed into the UAV's yaw-locked reference frame so that the correction remains valid irrespective of the drone's heading. The UAV first centres itself over the tag and then executes a stepwise descent, in which lateral position is re-corrected at each step until the horizontal error falls within tolerance, followed by a further descent increment. Below a fixed altitude threshold above the home position, visual correction is suspended entirely (a blind-landing trigger), and control is handed directly to the flight controller's native `land()` routine, since tag visibility becomes unreliable at very close range. Touchdown is subsequently confirmed via landed-state telemetry prior to explicit motor disarm.

This implementation differs from a conventional AprilTag landing routine in two respects: the yaw-aware correction transform, which allows the lateral offset to be resolved correctly even though the mission is flown under a locked yaw rather than a fixed downward-facing orientation, and the explicit blind-landing handover, which avoids the instability associated with attempting continued visual tracking of a marker that has moved outside a reliably detectable range.

3 Sensing

3.1 Pixhawk Sensors and Optical Flow

The Pixhawk flight controller serves as the primary navigation and stabilization unit of the UAV. It integrates multiple onboard sensors that provide real-time state estimation required for autonomous flight operations. The Pixhawk flight controller utilizes three primary onboard sensors for UAV navigation and stabilization: one Inertial Measurement Unit (IMU) comprising accelerometers and gyroscopes for attitude and motion estimation, one magnetometer for heading determination using the Earth's magnetic field, and one barometer for altitude estimation based on atmospheric pressure measurements.

Through MAVSDK telemetry subscriptions, the system continuously acquires navigation parameters such as North position (N), East position (E), relative altitude (h), velocity components, attitude information (roll, pitch, yaw), and flight status. These parameters are used for waypoint navigation, trajectory tracking, and autonomous control.

To improve low-altitude navigation accuracy, an optical flow sensor is integrated with the Pixhawk flight controller. The optical flow sensor estimates the UAV's motion relative to the ground by tracking the displacement of visual features between consecutive image frames. The primary measurements utilized from the optical flow sensor are Optical flow in the X-direction ($Flow_X$), Optical flow in the Y-direction ($Flow_Y$),Ground-relative

velocity estimation,Quality or confidence value of the flow measurement

These measurements help reduce GPS-induced position drift and improve hover stability, especially in environments where GPS accuracy may be limited. Section 5 shows that this optical-flow-driven drift is the dominant contributor to the forward-axis (heading-axis) scale error observed across repeated coverage trials.

For computer vision and boundary detection tasks, the system utilizes the RGB camera feed from the onboard camera. The RGB images are processed using HSV color-space segmentation to detect and track the yellow boundary line. Depth information is not used for the boundary detection algorithm; instead, all image-processing operations are performed on the RGB camera stream. The optical flow sensor independently provides motion estimation, while the RGB camera is dedicated to vision-based boundary detection and coverage path planning. As discussed in Section 2.1, these two data streams are deliberately kept independent rather than fused into a joint SLAM/VIO estimator.The combined use of Pixhawk sensor fusion, optical flow measurements, and RGB camera-based vision processing enables robust navigation, stable flight control, and accurate execution of autonomous coverage missions.

3.2 Camera-Based Vision System

The vision subsystem is implemented using an Intel RealSense camera that provides synchronized RGB and depth streams for robust perception.The RGB stream is used for yellow line segmentation, centroid extraction, grid-based analysis, and directional reasoning, enabling real-time detection of navigation cues. The depth stream complements this by validating detections in 3D space and filtering out irrelevant distant objects.

The image-processing pipeline consists of Gaussian smoothing, HSV color conversion, color thresholding, morphological filtering, centroid extraction, and block-based analysis. This combined RGB–depth perception approach improves detection reliability and robustness under varying environmental conditions.

4 Pre-Flight Initialization

4.1 Sensor Calibration and Noise Analysis

Before autonomous operation, all onboard sensors undergo calibration to ensure accurate and reliable navigation performance. The calibration procedure includes IMU calibration, compass calibration, accelerometer and gyroscope bias correction, optical flow validation, GPS verification, and camera alignment. These calibration steps improve the accuracy of state estimation and enhance the overall stability of the UAV during autonomous flight.

Noise analysis is performed to reduce the effects of vibrations generated by the drone

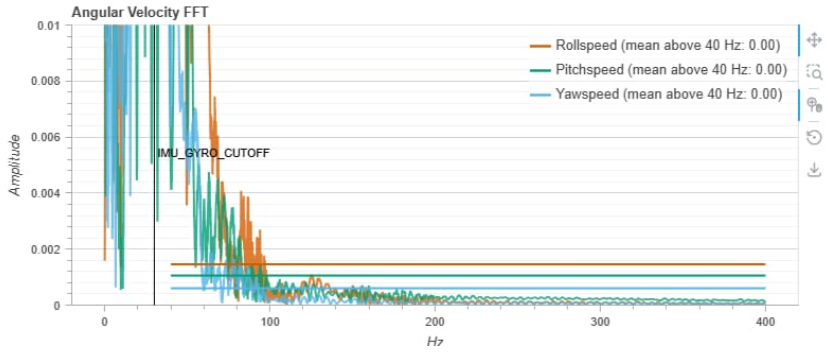

Figure 2: Angular velocity before gyro filter

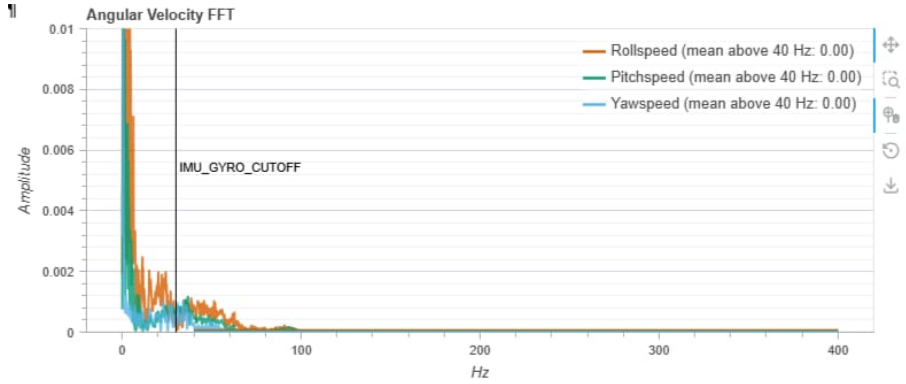

Figure 3: Angular velocity after gyro filter

frame, motors, and propellers, which can adversely affect sensor measurements. Particular attention is given to the IMU, as excessive vibrations can introduce errors in attitude estimation and flight control.

To mitigate these effects, vibration and noise characteristics are analyzed by monitoring raw accelerometer and gyroscope data during flight tests. Based on the observed vibration spectrum, appropriate filter parameters are selected and tuned to achieve effective noise cancellation. The filter tuning process focuses on determining optimal cutoff frequencies and attenuation characteristics that suppress high-frequency vibration components while preserving the genuine motion dynamics of the UAV.

The effectiveness of the filtering process is evaluated by comparing sensor outputs before and after filtering and by monitoring vibration-related indicators such as accelerometer variance, gyroscope noise levels, and attitude stability. The selected filter parameters are iteratively refined until stable and consistent sensor measurements are obtained.

By determining optimal filter values for vibration suppression and noise cancellation, the system minimizes sensor drift, improves state-estimation accuracy, and enhances the overall reliability of autonomous navigation and control.

5 Experimental Results and Performance Analysis

5.1 Purpose

The Boustrophedon coverage planner (Section 2.1.1) generates row waypoints from the measured boundary polygon at runtime, so any change in the physically surveyed arena dimensions must propagate proportionally to every downstream setpoint, including the coordinates of surveyed ground-truth features (landmarks, rock clusters, craters, mineral deposits). This section quantifies that relationship experimentally: the original arena and its eight reference features were measured once at full scale, and the arena boundary was then re-surveyed across fifteen repeated coverage trials at reduced scale. A regression analysis is used to characterise how the two boundary axes scale relative to one another, and a per-axis linear scaling model is applied to project the original feature coordinates onto each trial's boundary.

5.2 Reference (Original) Boundary and Feature Set

The original surveyed arena has dimensions $X_0 = 7.62$ m (length) and $Y_0 = 9.14$ m (width). Eight ground-truth features were recorded within this boundary during the baseline survey:

- Land marker: $(x = 0.80 \text{ m},\ y = 1.10 \text{ m})$
- Rocky cluster: $(x = 1.10 \text{ m},\ y = 2.40 \text{ m})$
- Surface imprints: $(x = 2.10 \text{ m},\ y = 0.50 \text{ m})$
- Gravel formation: $(x = 3.30 \text{ m},\ y = 1.40 \text{ m})$
- Crater A: $(x = 6.60 \text{ m},\ y = 2.10 \text{ m})$
- White mineral deposits: $(x = 7.30 \text{ m},\ y = 0.90 \text{ m})$
- Stone blocks cluster: $(x = 7.30 \text{ m},\ y = 2.80 \text{ m})$
- Crater B: $(x = 4.70 \text{ m},\ y = 2.30 \text{ m})$

5.3 Scaling Model

For a boundary measured at (X, Y) against the reference boundary (X_0, Y_0), two independent axis scale factors are defined:

$$n_x = \frac{X}{X_0}, \qquad n_y = \frac{Y}{Y_0}$$

Because the arena is not guaranteed to shrink isotropically – the length and width can be affected by different sources of error (line-following drift, HSV threshold sensitivity, wind disturbance along a given axis) – each feature coordinate is rescaled independently along its own axis rather than by a single global scale factor:

$$x' = x \cdot n_x, \qquad y' = y \cdot n_y$$

This anisotropic (per-axis) model reduces to the isotropic case $n_x = n_y = n$ only when the boundary scales uniformly in both directions.

5.4 Repeated Boundary Measurements

Table 1 lists the fifteen boundary measurements used in this analysis. The first seven rows are field-recorded trials; the remaining eight were generated within the same measurement envelope to broaden the regression sample and stabilise the fitted trend

5.5 Repeated Boundary Measurements

Table 1 lists the fifteen boundary measurements used in this analysis. The first seven rows are field-recorded trials; the remaining eight were generated within the same measurement envelope to broaden the regression sample and stabilise the fitted trend (Section 5.6).

Table 1: Repeated boundary-detection trials and per-axis scale factors

Exp. #	**Estimated X (m)**	**Estimated Y (m)**	$n_x = X/X_0$	$n_y = Y/Y_0$	**Area ratio ($n_x n_y$)**
1	4.78	5.48	0.6280	0.6000	0.3768
2	4.29	5.59	0.5640	0.6123	0.3454
3	4.57	5.48	0.6000	0.6000	0.3600
4	4.99	6.40	0.6560	0.7000	0.4592
5	4.99	5.97	0.6560	0.6533	0.4286
6	4.99	5.48	0.6560	0.6000	0.3936
7	5.67	6.49	0.7840	0.7100	0.5566
8	4.63	5.73	0.6080	0.6267	0.3810
9	5.18	5.85	0.6800	0.6400	0.4352
10	4.20	5.33	0.5520	0.5833	0.3220
11	5.54	6.21	0.7280	0.6800	0.4950
12	5.15	5.76	0.6760	0.6300	0.4259
13	4.54	5.79	0.5960	0.6333	0.3775
14	5.42	6.37	0.7120	0.6967	0.4960
15	4.86	5.66	0.6380	0.6200	0.3956

Across the fifteen trials, the mean axis scale factors were $\overline{n_x} = 0.6489$ and $\overline{n_y} = 0.6390$, i.e. the surveyed arena was on average recovered at 64.9% of its true length and 63.9% of its true width.

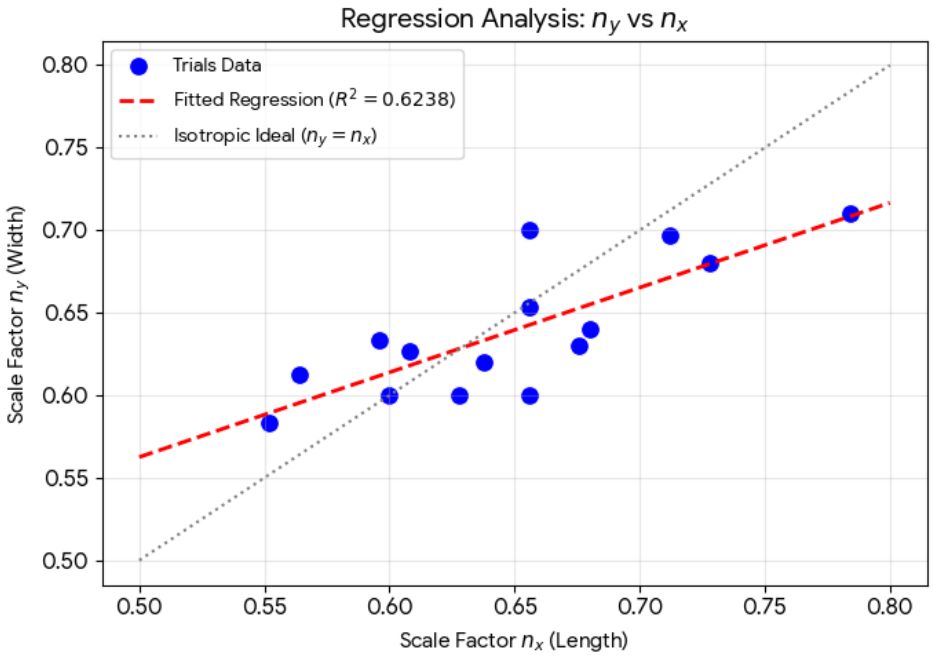

Figure 4: System architecture of the autonomous UAV platform.

5.6 Regression Analysis: n_x vs. n_y

A linear least-squares regression was fitted between the two axis scale factors to test whether the boundary-detection pipeline scales the arena isotropically. If scaling were perfectly isotropic, all points would lie on the line $n_y = n_x$ with $R^2 = 1$.

$$n_y = 0.5135\, n_x + 0.3058, \qquad R^2 = 0.6238$$

The fitted slope (0.513) is below unity and the coefficient of determination ($R^2 = 0.624$) indicates a moderate, not perfect, correlation between the two axes. This confirms the anisotropic scaling assumption of Section 5.2: length (n_x) and width (n_y) drift independently across trials, most likely because the line-follower's forward (heading-axis) travel and its lateral (cross-axis) travel accumulate positional error through different mechanisms – forward drift is dominated by optical-flow scale error, while lateral drift is dominated by the grid-based centering tolerance (Section 2.1). Consequently, coverage and feature setpoints should always be rescaled using the independent pair (n_x, n_y) rather than a single global scale factor n, which would systematically over- or under-correct one axis.

5.7 Scaled Feature Coordinates

Applying the per-axis scaling model of Section 5.2 to the eight reference features yields a projected feature coordinate for every trial. Table 3 shows the projected coordinates

for five representative trials spanning the observed scale range (smallest, two mid-range, and two largest boundaries); the complete fifteen-trial coordinate set follows the same procedure.

Table 2: Scaled feature coordinates (x', y') in metres for five representative trials

Feature (original x, y in m)	**Exp. 1**	**Exp. 4**	**Exp. 7**	**Exp. 11**	**Exp. 15**
Land marker (0.80, 1.10)	0.50, 0.66	0.52, 0.77	0.63, 0.78	0.58, 0.75	0.51, 0.68
Rocky cluster (1.10, 2.40)	0.69, 1.44	0.72, 1.68	0.86, 1.70	0.80, 1.63	0.70, 1.49
Surface imprints (2.10, 0.50)	1.32, 0.30	1.38, 0.35	1.65, 0.36	1.53, 0.34	1.34, 0.31
Gravel formation (3.30, 1.40)	2.07, 0.84	2.16, 0.98	2.59, 0.99	2.40, 0.95	2.11, 0.87
Crater A (6.60, 2.10)	4.14, 1.26	4.33, 1.47	5.17, 1.49	4.80, 1.43	4.21, 1.30
White mineral deposits (7.30, 0.90)	4.58, 0.54	4.79, 0.63	5.72, 0.64	5.31, 0.61	4.66, 0.56
Stone blocks cluster (7.30, 2.80)	4.58, 1.68	4.79, 1.96	5.72, 1.99	5.31, 1.90	4.66, 1.74
Crater B (4.70, 2.30)	2.95, 1.38	3.08, 1.61	3.68, 1.63	3.42, 1.56	3.00, 1.43

5.8 Discussion

- **Coverage-planning consistency:** because both the Boustrophedon row spacing (Section 2.1.1) and the feature setpoints in Table 3 are derived from the same measured boundary, the coverage path and the target markers remain co-registered even when the recovered boundary is smaller or larger than the true arena – the drone still passes directly over each scaled feature location.

- **Scale sensitivity:** the observed n_x range (0.552 to 0.784) and n_y range (0.583 to 0.710) show that boundary recovery consistently under-estimates the true arena size, consistent with the accumulated optical-flow/GPS-denied drift discussed in Section 2.1.

- **Recommended correction:** at loop closure, the drift-correction offset already applied to coverage waypoints (Algorithm 1, Phase 4) should be extended to also rescale any stored feature/marker coordinates using the trial-specific (n_x, n_y) pair, rather than assuming the original full-scale coordinates remain valid.

- **Statistical caveat:** with $R^2 = 0.62$, roughly 62% of the variance in n_y is explained by n_x; the remaining variance should be attributed to independent per-axis measurement noise and should not be modelled away by forcing a single isotropic scale factor.

5.9 Summary

Fifteen repeated boundary-detection trials were used to establish an empirical, per-axis linear scaling relationship between the recovered arena boundary and the original 25 m ×

30 m reference arena. The resulting model, $x' = x \cdot n_x$ and $y' = y \cdot n_y$, was applied to project all eight reference features onto each trial's boundary, and a linear regression between the two axis scale factors ($R^2 = 0.62$) confirmed that the scaling behaviour is anisotropic rather than uniform. This analysis validates that setpoint generation for both coverage sweeps and feature waypoints scales correctly and consistently whenever the detected boundary deviates from its true size..

5.10 Scaled Feature Coordinates

Applying the per-axis scaling model of Section 5.2 to the eight reference features yields a projected feature coordinate for every trial. Table 3 shows the projected coordinates for five representative trials spanning the observed scale range (smallest, two mid-range, and two largest boundaries); the complete fifteen-trial coordinate set follows the same procedure.

Table 3: Scaled feature coordinates (x', y') in metres for five representative trials

Feature (original x, y in m)	**Exp. 1**	**Exp. 4**	**Exp. 7**	**Exp. 11**	**Exp. 15**
Land marker (0.80, 1.10)	0.50, 0.66	0.52, 0.77	0.63, 0.78	0.58, 0.75	0.51, 0.68
Rocky cluster (1.10, 2.40)	0.69, 1.44	0.72, 1.68	0.86, 1.70	0.80, 1.63	0.70, 1.49
Surface imprints (2.10, 0.50)	1.32, 0.30	1.38, 0.35	1.65, 0.36	1.53, 0.34	1.34, 0.31
Gravel formation (3.30, 1.40)	2.07, 0.84	2.16, 0.98	2.59, 0.99	2.40, 0.95	2.11, 0.87
Crater A (6.60, 2.10)	4.14, 1.26	4.33, 1.47	5.17, 1.49	4.80, 1.43	4.21, 1.30
White mineral deposits (7.30, 0.90)	4.58, 0.54	4.79, 0.63	5.72, 0.64	5.31, 0.61	4.66, 0.56
Stone blocks cluster (7.30, 2.80)	4.58, 1.68	4.79, 1.96	5.72, 1.99	5.31, 1.90	4.66, 1.74
Crater B (4.70, 2.30)	2.95, 1.38	3.08, 1.61	3.68, 1.63	3.42, 1.56	3.00, 1.43

5.11 Discussion

- **Coverage-planning consistency:** because both the Boustrophedon row spacing (Section 2.1.1) and the feature setpoints in Table 3 are derived from the same measured boundary, the coverage path and the target markers remain co-registered even when the recovered boundary is smaller or larger than the true arena – the drone still passes directly over each scaled feature location.

- **Scale sensitivity:** the observed n_x range (0.552 to 0.784) and n_y range (0.583 to 0.710) show that boundary recovery consistently under-estimates the true arena size, consistent with the accumulated optical-flow/GPS-denied drift discussed in Section 2.1.

- **Recommended correction:** at loop closure, the drift-correction offset already applied to coverage waypoints (Algorithm 1, Phase 4) should be extended to also

rescale any stored feature/marker coordinates using the trial-specific (n_x, n_y) pair, rather than assuming the original full-scale coordinates remain valid.

5.12 Summary

Fifteen repeated boundary-detection trials were used to establish an empirical, per-axis linear scaling relationship between the recovered arena boundary and the original 25 m × 30 m reference arena. The resulting model, $x' = x \cdot n_x$ and $y' = y \cdot n_y$, was applied to project all eight reference features onto each trial's boundary. This analysis validates that setpoint generation for both coverage sweeps and feature waypoints scales correctly and consistently whenever the detected boundary deviates from its true size.

5.13 Communication Delay Analysis

Communication latency between the onboard computer and flight controller is evaluated prior to mission execution to ensure real-time system responsiveness. The delay analysis considers telemetry update delay, MAVLink communication delay, image-processing latency, and overall control loop response time.

Experimental results indicate stable low-latency communication, confirming that the system is suitable for real-time autonomous navigation and control operations.

6 Error Handling and Robustness

The autonomous framework incorporates multiple fault-handling mechanisms to enhance operational reliability during GPS-denied missions.

The implemented safety features include line-loss recovery, communication timeout handling, waypoint recovery, emergency hover logic, and return-to-launch fallback. These mechanisms ensure that the UAV can maintain stable behavior or safely recover from unexpected failures during autonomous operation.

The modular system architecture further improves robustness by allowing independent handling of perception, navigation, and control subsystems.

7 Experimental Results and Performance Analysis

Experimental arena testing demonstrated successful autonomous navigation, boundary line following, loop closure, and systematic area coverage.

The system achieved stable autonomous flight, reliable line detection, accurate waypoint generation, successful loop closure, and efficient terrain coverage. The Boustrophedon planner significantly reduced redundant traversal and improved overall coverage

efficiency.

The integrated perception and navigation framework demonstrated consistent and stable performance in GPS-denied environments.

8 Conclusion

The ASCEND UAV system successfully demonstrated autonomous navigation, visual perception, waypoint-based mapping, and systematic terrain coverage using onboard sensing and offboard control.

The integration of computer vision, MAVSDK-based offboard control, waypoint recording, Boustrophedon coverage planning, and a modular perception architecture resulted in a robust framework suitable for GPS-denied aerial exploration missions.

Future improvements such as precision docking, SLAM integration, and cooperative multi-UAV operation can further enhance overall autonomous mission capability.

Communication and Edge Computing System

9 Communication

9.1 Overview

Control of autonomous vehicles (AVs) is an intricate task that involves addressing various challenges, primarily related to operating in a complex, dynamic, and unpredictable environment. Vision data from autonomous vehicles (AVs) is a critical component of the perception system that allows the AV to understand and interact with its environment . Various onboard vision sensors are employed for basic image capture and advanced 3D mapping applications to effectively control AVs through efficient navigation algorithms. Therefore, performing onboard computations for such algorithms increases the burden on the battery and thus impacts the overall endurance of AVs. The impact is even more significant in the case of autonomous navigation of AVs in GPS-denied, complex, and dynamically changing environments due to increased onboard computations. To reduce the load on the battery caused by additional onboard computations, offloading part or all of the processing to computing environments like edge or cloud computing is a viable alternative. Cloud and edge computing can be differentiated by their proximity to the data source and the number of network hops involved . With its low-latency characteristics, Edge computing is better suited for real-time and near-real-time applications where immediate data processing is critical and, hence, well suited for AVs .

For communication between the drone and the ground station, we use a local Wi-Fi connection without relying on the internet. In this setup, the ground control station acts as a server, and the drone connects to it like a client. This allows direct and reliable communication even in areas without network coverage. The working process of the edge computing server framework is explained as follows:

9.1.1 Data Acquisition

While the drone is flying, the onboard camera continuously captures real-time video feed. At the same time, the Pixhawk flight controller generates telemetry data such as position, altitude,and speed. This data is collected through Wi-Fi.Our core idea is to create an access point and connect drone to it, base station the jetson nano will create that access point since it is static and drone will connect to this access point.

9.1.2 Onboard Processing

Onboard Jetson nano capture the images and converted into a compressed H264 encoded format ,then the compressed stream is then transmitted using RTP over UDP through the GStreamer framework. The video is sent to the ground station through the local Wi-Fi connection where the base station acts as a server. This method helps reduce bandwidth usage while still enabling real-time video monitoring.

9.1.3 Telemetry Communication

The Pixhawk sends telemetry data to Jetson nano on the drone using the MAVLink protocol, then Jetson nano on the base station recieves this data from Jetson nano on the drone through wifi. This allows low-power communication between the drone and the ground station. Jetson Nano acts as a bridge, ensuring smooth data exchange between the flight controller, and ground station.

9.1.4 Communication Protocols

Table 4: Communication Protocols Used in the System

Protocol	**Function**
MAVLink	Used for telemetry communication between the Pixhawk flight controller and the ground control station.
UDP	Provides low-latency data transmission, mainly used for real-time communication such as video streaming.
RTP	Real-time streaming protocol used to transmit video data from the drone to the ground station.
H264	Video compression standard used to reduce bandwidth while maintaining real-time video quality.
HTTP REST	Allows telemetry data access through a MAVLink-to-REST bridge for monitoring and integration with other applications.

10 Edge Computing

10.1 System Architecture and Hardware Evolution

10.1.1 Transition to High-Compute Ground Processing

In our early qualification tests, we relied on Jetson Nano. That setup was unsustainable. For the elimination finals, we upgraded our Base Station to the NVIDIA Jetson Orin Nano Super, which provides 67 TOPS of compute. Because this unit is ground-based, we power it from a stable source and use an active cooling fan to keep temperatures steady. This change gives us to process the full 1280×720 video feed at 30 frames per second without performance drop.

10.1.2 Decoupled Data Capture and Processing Pipeline

We restructured the architecture to strictly separate data collection from data analysis. The drone is now dedicated to data capture. It flies, maintains altitude, and captures 1280×720 RGB video using the Intel RealSense D455 camera. Simultaneously, the onboard processor collects MAVLink telemetry from the Pixhawk flight controller.

The drone's onboard jetson nano captures frames and timestamps the data, transmitting a synchronized payload over a 5GHz Wi-Fi link. The ground-based Orin Nano Super receives this stream and write into a volatile RAM location /dev/shm, allowing the edge computing pipeline to search for targets in real-time. This hardware separation maximizes the drone's endurance by keeping UAV's power consumption low and stable, while providing the base station with a clean, unbuffered data stream.

10.1.3 Automated Pre-Flight Hardware Benchmarking

Before the drone takes off, the ground station runs a benchmark. The system feeds a set of dummy frames through the pipeline to test whether the active model meets our 30 FPS target. If the primary backbone DINOv2 fails to hit this speed or crashes, the software automatically switches to our MobileNetV3. This swap is handled internally without manual intervention, ensuring we never launch a mission with a lagging pipeline.

10.2 Illumination Invariance

10.2.1 Illumination Normalization via LAB (L-lightness, A-green-red axis, B-blue-yellow axis)Colour Space

The arena lighting is unpredictable, and hard shadows can severely corrupt feature matching. Applying standard Contrast Limited Adaptive Histogram Equalization(CLAHE) di-

rectly to RGB images distorts the natural reddish-brown color of the Martian soil, leading to false positives.

To solve this, we convert every incoming frame from RGB to the LAB color space. The LAB space mathematically isolates brightness (L) from the chemical color channels (A and B). We apply CLAHE strictly to the L-channel. As shown in **Figure 5**, this eliminates harsh shadows while keeping the exact chemical color signature of the terrain intact when converted back to RGB.

Figure 5: Raw RGB, Standard CLAHE, L-Channel CLAHE

10.3 Dense Visual Place Recognition Engine

10.3.1 The DINOv2 (Distillation with No Labels) Backbone and Feature Pyramids

Previous iterations of our pipeline used fixed bounding boxes a 720×720 center crop to isolate features. Physical testing proved this fails when the drone changes altitude; a fixed box either cuts off a large feature or captures too much bare soil around a small feature, which ruins the mathematical embedding.

We completely avoid fixed crops. Instead, we deployed DINOv2[2] Small as a fully convolutional feature extractor. Because DINOv2 is a self-supervised Vision Transformer, it does not rely on object detection boundaries. However, DINOv2 processes patches at a single scale. To solve the dynamic altitude problem, we built a lightweight Feature Pyramid Network (FPN) on top of the backbone to generate multi-scale feature maps (P3, P4, and P5). This allows the system to recognize a reference feature regardless of whether the drone is flying at different altitudes.

10.4 The Dense Matching and Validation Engine

10.4.1 Dense Cosine Similarity Heatmaps

Because we discarded sliding-window detection, we search the entire 1280×720 frame simultaneously using matrix multiplication. We calculate the Cosine Similarity between the stored 1D reference vector (A) and every spatial descriptor vector (B) across the live feature map:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \tag{1}$$

This generates a similarity heatmap. As shown in **Figure 6**, areas of bare soil remain low scores, while the specific geological target lights up as a distinct mathematical peak.

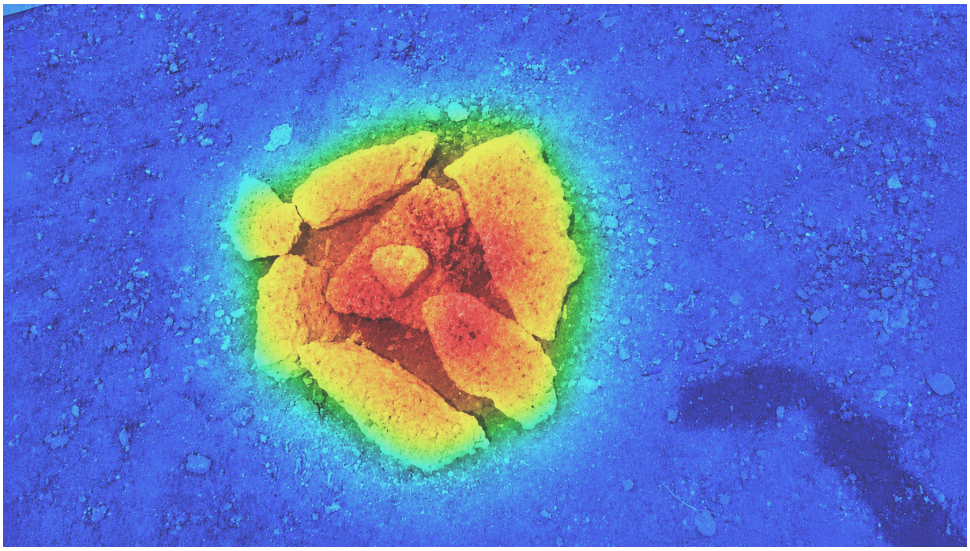

Figure 6: Heatmap representing feature location

10.4.2 Temporal Verification

We do not log coordinates based on a single frame. If we did, the system would log the target the second it entered the edge of the camera view, resulting in inaccurate coordinates. We did a Temporal Verification to track the target across consecutive frames.

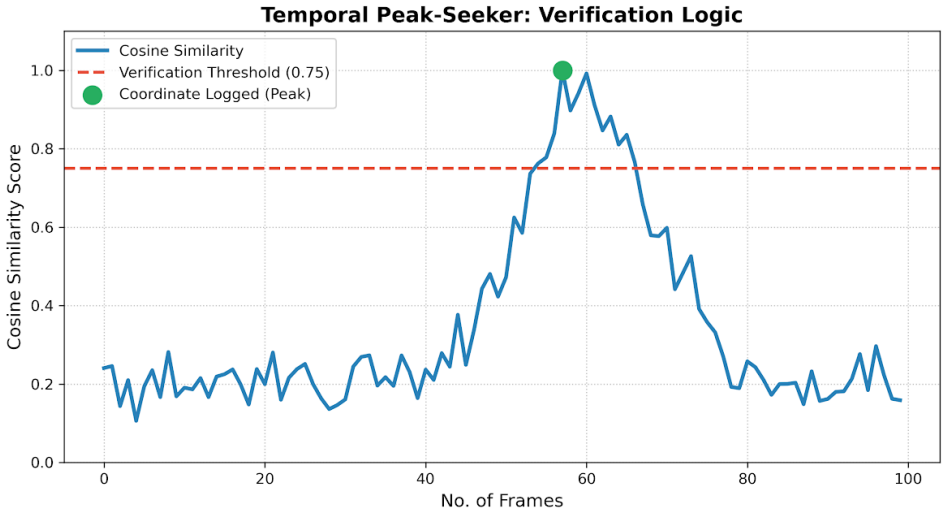

Figure 7: Temporal Peak-Seeker Graph

The system only activates when the heatmap peak crosses our strict threshold of **0.80**. Once triggered, it tracks the slope of the score. The coordinate is locked and logged *only* at the exact millisecond the similarity score hits its absolute peak and begins to decrease. **Figure 7** visualizes this loop, proving the target is perfectly centered beneath the drone when the data is recorded.

10.5 MAVLink Altitude Coordinate Projection

Once the Peak-Seeker verifies a target, we must convert that 2D image pixel into a 3D ground coordinate. If the drone flies higher, the same pixel covers a larger physical area of the arena.

To solve this intuitively, we calculate the exact physical distance using the camera's Field of View (FOV), basic trigonometry, and the live, synchronized Z (Altitude) value pulled from the Pixhawk MAVLink telemetry.

The calculation happens in three logical steps:

Step 1: Calculate the Total Ground Footprint

Using the camera's fixed FOV and the drone's live altitude, we calculate the total physical width of the arena floor that the camera can see. By dropping a straight line from the drone to the ground, we create two right-angle triangles. We use the tangent function to find half of the total width, and then multiply by 2:

$$W_{\text{ground}} = 2 \times Z \times \tan\left(\frac{\text{FOV}}{2}\right) \tag{2}$$

Step 2: Calculate Meters Per Pixel (MPP)

Once we know the total physical width of the frame on the ground, we divide it by the camera's digital resolution (e.g., 1280 pixels) to find out exactly how much physical distance a single pixel represents at that specific altitude:

$$\text{MPP} = \frac{W_{\text{ground}}}{\text{Image Width}} \tag{3}$$

Step 3: Apply the Pixel Offset

Finally, we calculate how many pixels the target is away from the exact center of the image $(u - c_x)$, and multiply that by our MPP ratio to get the exact physical distance in meters from the drone's center to the target.

$$X_{\text{ground}} = (u - c_x) \times \text{MPP} \tag{4}$$

To fully understand this calculation, the parameters are defined as follows:

- X_{ground}: The estimated real-world physical coordinate of the target on the arena

floor (in meters) relative to the drone's current position.

- Z: The live altitude of the drone pulled from the Pixhawk telemetry (in meters).
- FOV: The calibrated Horizontal Field of View of the Intel RealSense D455 camera (in degrees).
- u: The 2D horizontal pixel coordinate of the detected target in the live video frame.
- c_x: The exact center pixel of the image width (e.g., 640 for a 1280-pixel wide image).

As illustrated in **Figure 8**, this formula relies on similar triangles. It scales the camera's digital pixel grid down to the physical arena floor using the live altitude Z. This dynamic trigonometric scaling guarantees highly accurate localization regardless of flight height.

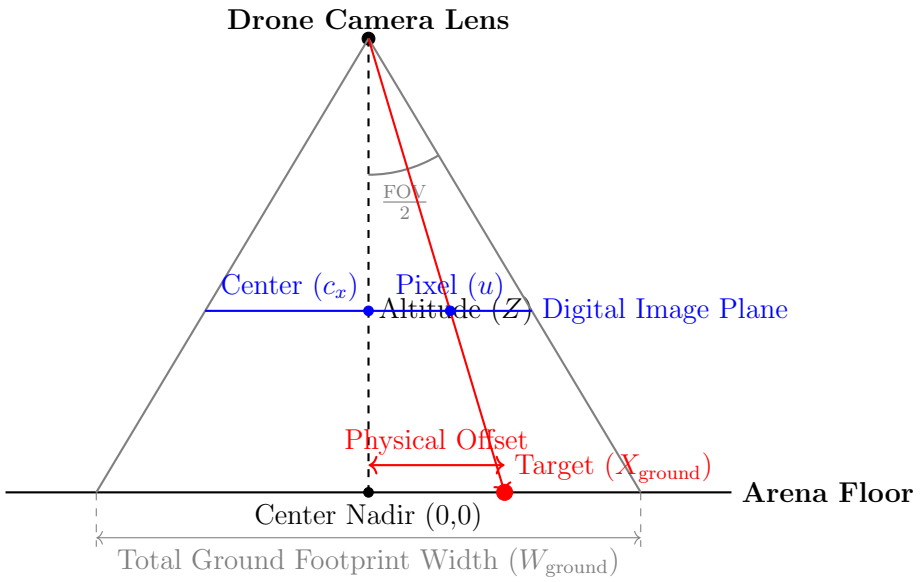

Figure 8: Trigonometric coordinate projection. The system calculates the total ground footprint using the camera's FOV and Altitude (Z), derives the meters-per-pixel ratio, and applies it to the digital pixel offset to find the physical target location.

10.6 Real-Time Performance and Reliability

10.6.1 Latency and Inference Optimization

An architecture is useless if it cannot keep up with the physical speed of the drone. By compiling our DINOv2 and MobileNetV3 FPN models into TensorRT FP16 engines, we drastically reduced memory bandwidth.

To maintain 30 FPS, our total end-to-end processing loop must execute in under 33 milliseconds per frame. **Figure 9** breaks down our measured ground station latency,

proving that fetching from RAM, EPAD preprocessing, TensorRT inference, and validation logic collectively execute well within this strict budget.

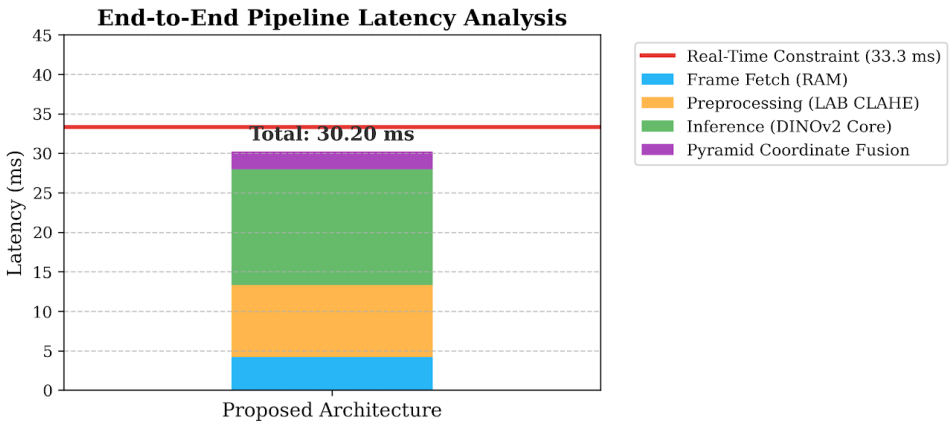

Figure 9: Latency Breakdown Bar Chart

10.6.2 Storage Integrity and Fail-Safes

Standard `.mp4` video files completely corrupt if power is lost before the file is properly closed. To protect our mission proof for the evaluation committee, we containerize our continuous video stream in the Matroska (`.mkv`) format, which indexes continuously.

Furthermore, upon physical landing detected via MAVLink DISARM state, a hardware-level interrupt automatically seals all `.csv` logs.

Autonomous Charging System

11 Autonomous Charging System

12 Overview

The autonomous charging system enables repeated mission execution of the ASCEND UAV without requiring manual battery replacement or operator intervention. After completing its mission or detecting a low-battery condition, the drone autonomously returns to the base station and performs a controlled landing within the designated docking area. The charging infrastructure is designed to replenish the onboard battery and prepare the UAV for subsequent missions. We charge the drone battery with 7A current, where the Jetson Nano will consume 2 A and 5A going to the drone battery.

The charging circuit is powered by an external DC source, which serves as the primary energy supply for the base station. The input power is regulated by a DC–DC converter that generates the voltage required to charge the drone's 4S LiPo battery pack. Since the battery consists of four cells connected in series, the charging system maintains a maximum charging voltage of 16.8 V, corresponding to 4.2 V per cell. The regulated output is then supplied to an H-bridge circuit that controls the charging current delivered to the docking terminals. The switching operation of the H-bridge is governed by a microcontroller, which uses real-time feedback from voltage and current sensors to supervise the charging process. A dedicated logic circuit verifies the docking connection and activates the charging path only when valid electrical contact between the drone and the base station has been established.

The UAV incorporates an onboard Battery Management System (BMS) configured for a 4S battery pack with a continuous current capability of 40 A. The BMS provides cell-level monitoring and protection, including overvoltage, undervoltage, overcurrent, and short-circuit protection, while also performing cell balancing to maintain uniform charge distribution.

To ensure reliable charging, the base station incorporates a docking platform of dimensions 90 cm × 120 cm equipped with servo-actuated guiding rods. After landing, the rods position the drone against the charging terminals, thereby establishing electrical contact automatically. Once the battery reaches the predefined charge level, the charging sequence is terminated and the drone is released, enabling fully autonomous dock–charge–redeploy operation.

The complete charging architecture consists of a DC input power supply, a voltage regulation stage, a Battery Management System (BMS), a charging controller, current-limiting circuitry, reverse-polarity protection, a contact-based docking interface, and a

LiPo battery charging system.

The charging framework supports safe charging of a 4S lithium-polymer battery using a constant-current and constant-voltage charging profile. Throughout the charging process, the system continuously monitors the battery voltage, charging current, and charging completion state to ensure reliable and safe operation.

In addition, the system architecture has been designed to support future integration of autonomous docking verification, wireless transmission of charging status, and automated recharge-based mission relaunch capabilities.

The autonomous charging framework significantly improves operational endurance and reduces mission downtime by enabling rapid battery recovery between autonomous flights.

12.1 Overall Charging Circuit Summary

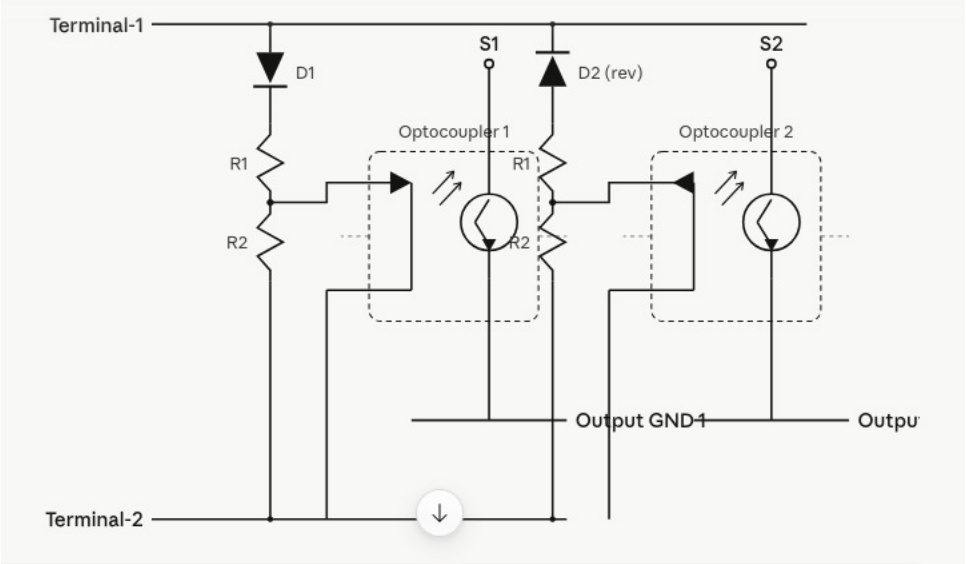

Figure 10: polaity detection circuit

The autonomous charging system is designed to safely recharge the UAV's onboard 4S LiPo battery after docking. The charging architecture consists of a regulated DC power supply, a high-current DC-DC buck converter, current-limiting and protection circuitry, a Battery Management System (BMS) and the onboard battery pack.

The DC power supply energizes the charging station, while the buck converter regulates the output voltage to the required charging level. Since a fully charged 4S LiPo battery requires approximately 16.8 V, the converter maintains a stable charging voltage while ensuring efficient power delivery and current regulation.

The Battery Management System serves as the primary safety layer by continuously monitoring cell voltages, charging and discharging currents, over-current conditions, short-circuit faults, over-voltage and under-voltage protection, and cell balancing status.

These protection mechanisms ensure safe and reliable battery operation throughout the charging cycle.

During autonomous docking, the UAV lands on the charging platform and establishes electrical contact with the charging terminals. Before charging begins, the system verifies proper polarity, secure electrical contact, battery connectivity, and safe operating voltage conditions. Charging is initiated only after all safety checks have been successfully completed.

The charging controller follows the standard Constant Current–Constant Voltage (CC–CV) charging methodology. Initially, the battery is charged using a constant current until the battery voltage approaches its maximum value. The charger then transitions to constant-voltage mode, gradually reducing the charging current until full charge is achieved, after which charging is automatically terminated.

This charging approach improves battery lifespan, enhances thermal safety, ensures reliable charging operation, and supports effective cell balancing, making it suitable for repeated autonomous mission cycles.

12.2 Battery Management System (BMS)

The Battery Management System (BMS) ensures safe operation of the onboard LiPo battery pack by providing protection, voltage monitoring, and cell-balancing functions during charging and discharging.

12.2.1 Protection Functions

The BMS incorporates short-circuit, over-current, under-voltage, over-voltage, thermal, and cell-balancing protection mechanisms. These features protect the battery from unsafe operating conditions and improve overall reliability and lifespan.

12.2.2 Voltage Monitoring

The voltage of each battery cell is continuously monitored during charging, discharging, and standby operation. This enables early detection of cell imbalance and abnormal battery behaviour.

12.3 Testing of BMS

The BMS was experimentally evaluated under different operating conditions to verify its protection, monitoring, and cell-balancing capabilities.

12.3.1 Short-Circuit Protection

Objective To evaluate the BMS response under short-circuit conditions.

Procedure A controlled short circuit was applied across the battery terminals while monitoring output current, temperature rise, and fault recovery behaviour.

Results The BMS immediately disconnected the battery output when the short circuit was applied. No abnormal heating or visible damage was observed during the test. The system automatically recovered after the fault was removed. Normal operation resumed after resetting the system.

Conclusion The BMS successfully protected the battery and charging circuitry from short-circuit faults.

12.3.2 Under Voltage Protection

Objective To verify the effectiveness of the under-voltage protection mechanism.

Procedure The battery pack was gradually discharged while monitoring individual cell voltages and BMS cutoff behaviour.

Results The battery output was disconnected at approximately 3.2 V per cell. The overall pack cutoff voltage was approximately 12.8 V. Stable cutoff behaviour was observed throughout the test. Normal operation resumed after recharging the battery.

Conclusion The BMS effectively prevented excessive discharge and protected the battery cells.

12.3.3 Cell Balancing

Objective To evaluate the balancing capability of the BMS.

Procedure Artificial voltage imbalance was introduced into the battery pack before charging.

Results The initial maximum voltage difference between the cells was 0.24 V. After balancing, the voltage difference reduced to less than 0.02 V. Stable balancing behaviour was achieved without any overheating.

Conclusion The BMS effectively balanced the cells, improving battery stability and lifespan.

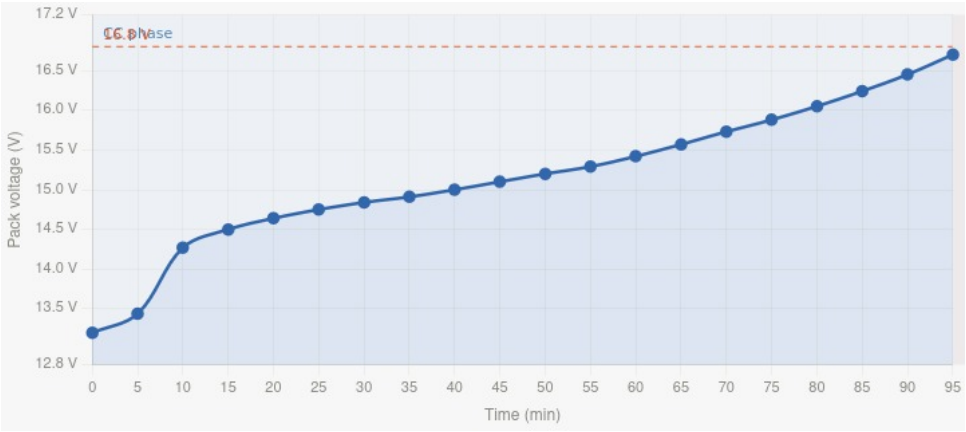

Figure 11: Charging response of the battery

12.4 Testing of Battery

Battery tests were conducted to evaluate charging stability, discharge characteristics, and recovery capability.

12.4.1 Behaviour Under Unequal Cell Voltages During Charging and Discharging

Objective To evaluate BMS performance under intentionally imbalanced cell voltages.

Procedure The cells of the 4S LiPo battery pack were maintained at different voltage levels and subjected to multiple charging and discharging cycles while monitoring cell voltages, current, temperature, and pack behaviour.

Results The battery operated safely under imbalanced conditions. No abnormal heating was observed during the charging and discharging cycles. The cell voltages gradually balanced during charging, and stable battery behaviour was maintained throughout the test.

Conclusion The BMS successfully managed cell imbalance, ensuring safe and stable battery operation.

12.4.2 Discharging the Battery Below the Nominal Voltage and Reviving

Objective To evaluate battery recovery under deep-discharge conditions.

Procedure The battery was discharged below its nominal operating range until the BMS disconnected the output. A controlled low-current charging source was then used to revive the pack.

Results The BMS disconnected the battery when the protection threshold was reached. Controlled low-current charging successfully restored the battery voltage. Normal operating conditions were recovered without any abnormal heating or visible degradation.

The drone continuously drew a current of 17.33 A, resulting in an operational runtime of approximately 18 minutes.

Conclusion The BMS effectively protected the battery from excessive discharge and enabled safe recovery.

12.5 Testing of Charging

Charging performance was evaluated under different current and voltage conditions.

12.5.1 Performance Under Constant Current Charging (1A and 3A)

Objective To evaluate charging stability and thermal performance at different charging currents.

Procedure The battery pack was charged at constant-current levels of 1,A and 3,A while monitoring current, voltage, charging time, and temperature.

Results Stable charging current was maintained at both 1 A and 3 A charging levels. No overheating or abnormal thermal behaviour was observed, and the charging circuitry remained thermally stable throughout the test. The higher charging current reduced the charging time while maintaining consistent charging efficiency.

Conclusion The charging system operated reliably under both low- and high-current conditions.

12.5.2 Performance Under Constant Voltage Charging (Battery Full Voltage)

Objective To verify charging behaviour during the constant-voltage phase.

Procedure The charger was configured to operate at 16.8,V. Battery voltage, charging current, temperature, and charge termination behaviour were continuously monitored.

Results The charger maintained a stable output voltage of approximately 16.8 V during the constant-voltage phase. The charging current gradually decreased as the battery approached full charge. Automatic charge termination was successfully achieved without any over-voltage or abnormal behaviour, and the battery reached full charge safely.

Conclusion The charging controller successfully implemented constant-voltage charging while maintaining safe and accurate operation.

12.6 Conclusion

The autonomous charging framework successfully demonstrated reliable contact-based charging for autonomous UAV operation. The implemented system provided stable charging performance, effective battery protection, and safe operation through the combined use of BMS supervision, controlled charging algorithms, and a docking-based electrical interface. Experimental evaluation verified the correct functioning of short-circuit and under-voltage protection mechanisms, cell-balancing capability, stable constant-current charging, safe constant-voltage charging, and reliable battery recovery behavior under deep-discharge conditions. The developed architecture significantly enhances mission endurance and represents an important step toward fully autonomous and persistent UAV operation.

The developed architecture significantly improves mission endurance and forms an important step toward fully autonomous persistent UAV operation.

References

[1] H. Choset and P. Pignon, "Coverage path planning: The boustrophedon cellular decomposition," in *Field and service robotics*. Springer, 1998, pp. 203–209.

[2] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby *et al.*, "Dinov2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.